

INSTRUCCIONES DE EJECUCIÓN TEST ESTRUCTURA

🚀 Aquí tienes un script en Python listo para producción, modular y compatible con cualquier runtime LLM que soporte API compatible con OpenAI (Ollama, LM Studio, OpenRouter, Groq, OpenAI, etc.).

Este script:

- ✓ Verifica que los 4 archivos base existen en `.ai/`
 - ✓ Envía los prompts de prueba de forma automática
 - ✓ Evalúa respuestas por keywords y exclusiones
 - ✓ Genera un reporte en consola + JSON
 - ✓ Es fácil de adaptar a tu stack real
-

📦 `test_ai_structure.py`

python

🔧 Cómo usarlo

1. Instalar dependencia:

```
pip install openai
```

2. Configurar variables de entorno (ajusta a tu runtime):

```
export LLM_BASE_URL="http://localhost:11434/v1" # Ollama
```

```
# o export LLM_BASE_URL="https://api.openai.com/v1"
```

```
export LLM_API_KEY="sk-tu-key-aqui"
```

```
export LLM_MODEL="qwen2.5-coder:32b" # o gpt-4o, claude-3-5, etc.
```

3. Ejecutar:

```
python test_ai_structure.py
```

🔍 ¿Cómo funciona la validación?

- No revisa archivos en disco, sino el comportamiento real del agente tras inyectar `.ai/` en su prompt de sistema.
- Usa `temperature=0.1` para respuestas deterministas y predecibles.
- Evalúa con regex de límites de palabra (`\b`) para evitar falsos positivos (ej: `any` vs `analysis`).

- Si tu runtime no soporta API OpenAI-compatible, puedes adaptar la función `run_test()` para usar `subprocess` + CLI de tu herramienta.
-

Personalización rápida

- Ajusta `expected_contains` en el Test 3 para que coincidan exactamente con tu `context.md` (ej: `["next.js", "vercel", "tRPC"]` si es tu stack).
- Agrega tests copiando un bloque de `TESTS` y cambiando `prompt`/keywords.
- Integra CI/CD: añade este script a un workflow de GitHub Actions para validar `.ai/` en cada PR.